

Kapitel 14. Klasser

Innehållsförteckning

[Allmänt om objektorientering](#)

[En första klass](#)

[Konstruktör och destruktör](#)

[Nästlade klasser](#)

Detta kapitel behandlar grunderna i objektorientering i C++. Det vi hittills behandlat av C++ har till största delen varit ämnen som fungerar på samma sätt i standard C (med några undantag).

Allmänt om objektorientering

De centrala begreppen inom objektorientering är *klass* och *objekt*. Det kan vara värt att redogöra för hur dessa relaterar sig till varandra för att undvika missförstånd i framtiden.

En *klass* definierar en datatyp i C++. Man kan efter att en klass definierats skapa variabler av denna datatyp. Klasser innehåller data och metoder, men en klass i sig kan inte göra något, den endast definierar hur variablerna skall se ut, på samma sätt som `struct`.

Variabler som har en viss klass som datatyp kallas *objekt*. Objekten innehåller egentlig data, och man kan kalla på metoder som objekt har. Man säger att ett objekt är en *instans av en viss klass*. Då man skapar ett objekt *instantierar* man en klass.

Abstraktion

Ett av de centrala målen med objektorientering är *abstraktion*. Man vil dölja komplexitet hos någonting genom att kapsla in fenomenet i en klass. Människan vill alltid ha abstraktioner för olika ting. Vi vill t.ex. inte se på en hand som en samling blod, ben, senor och kött, inte heller som en enorm samling celler av olika typ och allra minst som en ofantlig samling atomer. På samma sätt är vi nöjda över att slippa bry oss om våra program på disken som är en samling data, som är en samling bytes som är en samling bitar som är en samling magnetfält på en skiva. Klasser i C++ skall göra något komplext enkelt att förstå och använda.

Klasser är den mest avancerade abstraktionen i C++. Klasser låter oss abstrahera mycket komplex data på ett intuitivt sätt. Man kan bygga upp logiska arvshierarkier (se [Kapitel 15](#)) och kapsla in klasser i varandra. Traditionellt i C har programmeraren alltid vetat vad som finns "bakom" en datatyp, men i C++ behöver inte användaren längre veta detta. Tvärtom kan man totalt abstrahera bort implementationsdetaljer så att programmeraren inte ens kan se dessa, utan endast har ett väl definierat *gränssnitt* att jobba med. Ett gränssnitt definierar hur objekten kan användas.

Förutom endast abstraktion av data kan klasser även tillhandahålla *metoder* som opererar på det data ett objekt innehåller. På detta sätt abstraheras programmeraren från interna implementationsdetaljer angående data ett objekt innehåller, samt för hur denna data skall manipuleras för att åstadkomma ett önskat resultat.

Abstraktionen fungerar även åt andra hållet. Istället för att endast skydda programmeraren från

implementationsdetaljer används abstraktion även för att skydda objekt från programmeraren. På detta sätt kan man garantera att ett objekt alltid uppfyller vissa invarianter, eftersom programmeraren inte kan göra något annat med ett objekt än vad dess gränssnitt tillåter. Programmeraren ges inte tillträde till data inom ett objekt om det inte explicit tillåts. Man kan t.ex. tänka sig ett objekt som innehåller interna datastrukturer för ett användargränssnitt. Om programmeraren ges tillträde till alla dessa datastrukturer kan han/hon i misstag göra något fatalt som påverkar även andra programs funktioner negativt.

Återanvändning av kod

Ett av de andra centrala begreppen inom objektorientering är *återanvändning* av kod. C++ gör det lättare att skriva kod som kan återanvändas senare i andra projekt. Tack vare att implementationen är inkapslad behöver man inte veta hur någonting görs, utan det räcker med att veta att det görs korrekt för att klassen skall kunna återanvändas. En bra skapad klass är gjord så att dess funktionalitet kan modifieras via arvning (se [Kapitel 15](#)). Det kan kännas svårt för en nybörjare att skapa klasser som är "bra", men det är en teknik som man lär sig med tiden.

Andra fördelar med OO i C++

Objektorientering inom C++ är ett relativt komplext ämne. det finns väldigt mycket olika funktionalitet som i vissa fall kan verka ganska oförklarlig. Inom C++ kan man använda sig av de flesta av de fördelar modern objektorientering för med sig. Man kan t.ex. använda arvshierarkier för att skapa specialiserade klasser med gemensamma gränssnitt (se [Kapitel 15](#)). Man kan använda sig av typparametrisering för att åstadkomma generiska klasser (se [Kapitel 23](#)) som kan operera på olika datatyper. Överlagring av operatorer (se [Kapitel 21](#)) tillåter att klasser konstrueras att likna primitiva datatyper med samma typ av operatorer, eller göra dem lättare att använda. En funktionalitet som kallas exceptions (se [Kapitel 20](#)) tillåter avancerad rapportering av fel som gör program lättare att konstruera så att de är robusta. Numera är även Standard Template Library en del av standarden för C++. Den innehåller en stor mängd olika typparametriserade s.k. *containers* och algoritmer som i mycket stor grad gör det lättare att hantera olika typer av data i C++ (se [Kapitel 24](#)).

Objektorienterad design

Hur bra och ändamålsenliga klasser skapas är utanför detta kompendiums omfång. Mera information om detta finns i diverse litteratur om objektorientering. Se [Referenser](#) för referenser till litteratur. De klasser som demonstreras i detta kompendium är relativt enkla och inte alltid helt ändamålsenliga, men de existerar enbart för att visa någon viss aspekt inom C++. När man programmerar i C++ finns det inget som tvingar programmeraren att designa bra klasser, C++ ger endast verktygen för detta. Bra design är något man måste lära sig via försök och misstag. Ju mer man programmerar C++ desto bättre lär man sig visualisera olika klasser för olika entiteter i ett program.

[Förutgående](#)

Använda multipla filer

[Hem](#)

[Nästa](#)

En första klass